

Отчёт по практической работе

Дисциплина: Deep Learning for Computer Vision

Проект: Модернизация DeepSORT — отслеживание людей на видео

1. Введение

Задача множественного отслеживания объектов (MOT, Multiple Object Tracking) состоит в следующем: дано видео, требуется для каждого объекта (в данном случае — человека) присвоить уникальный идентификатор и отслеживать его перемещение во времени. Результат — набор траекторий, каждая из которых представляет собой последовательность ограничивающих прямоугольников (bbox, bounding box) с общим идентификатором.

Практическое применение MOT охватывает системы видеонаблюдения, спортивную аналитику, беспилотный транспорт и анализ скоплений людей. Основная сложность задачи — необходимость связывать обнаружения одного и того же объекта между кадрами при наличии окклюзий (взаимного перекрытия), схожей внешности разных объектов и вариативности масштаба.

В качестве базового алгоритма выбран DeepSORT (2017) — расширение метода SORT, добавляющее к модели движения (фильтр Калмана) анализ внешнего вида через модель повторной идентификации (REID, Re-Identification). REID-модель извлекает из каждого обнаруженного объекта числовой вектор — дескриптор внешности; сходство дескрипторов используется для ассоциации между кадрами.

Оригинальный DeepSORT использует устаревшие компоненты: детектор POI (Pedestrian Occupancy Information) и REID-модель mars-small128, реализованную на TensorFlow 1.x. В условиях данного проекта TensorFlow-модель недоступна, что делает компонент анализа внешности неработоспособным.

Цель работы — модернизация DeepSORT путём замены детектора и REID-модели на современные нейросетевые архитектуры с последующей настройкой параметров. Дополнительно реализована система автономного распознавания личностей (body REID) с базой данных дескрипторов и временным голосованием.

Работа выполнена в шесть этапов: 1. Фиксация базовой линии 2. Интеграция и сравнение детекторов 3. Интеграция и сравнение REID-моделей 4. Объединение в полный пайплайн, настройка параметров 5. Реализация автономного body REID 6. Генерация оверлей-видео, составление отчёта

2. Описание задачи

2.1. Постановка задачи

Формально: дано видео из N кадров, на каждом кадре необходимо обнаружить всех людей и назначить каждому уникальный идентификатор (ID), сохраняющийся во времени. Выходные данные — файл в формате MOT: каждая строка содержит номер_кадра, ID, x , y , ширина, высота, где (x, y) — координаты левого верхнего угла bbox (bounding box — ограничивающего прямоугольника).

2.2. Факторы, усложняющие задачу

- **Окклюзии** — взаимное перекрытие объектов приводит к временной потере наблюдения; алгоритм должен восстанавливать ID после повторного появления.
- **Сходство внешности** — дескрипторы REID для разных объектов могут быть близки в пространстве признаков.
- **Вариативность масштаба** — один и тот же объект может занимать от 10 до 500 пикселей в зависимости от расстояния до камеры.
- **Быстрое движение** — перемещение объекта между кадрами может превышать предсказание фильтра Калмана.

2.3. Метрики оценки

NOTA (Higher Order Tracking Accuracy) — основная метрика. Представляет собой среднее геометрическое двух компонент: - DetJ (Detection Jaccard — качество детекции) — мера сходства между обнаруженными и эталонными bbox (коэффициент Жаккара, то есть пересечение, делённое на объединение). - AssA (Association Accuracy — точность ассоциации) — качество связывания обнаружений одного объекта между кадрами.

У современных трекеров NOTA составляет 50-60. В данном проекте базовая линия — 4.73.

MOTA (Multiple Object Tracking Accuracy) — отношение числа корректных обнаружений минус ложноположительные (FP, false positives) и ложноотрицательные (FN, false negatives) к общему числу эталонных объектов. Измеряется в процентах; удовлетворительным считается значение 50+.

IDF1 (Identity F1) — гармоническое среднее точности и полноты идентификации. Низкий IDF1 указывает на частую смену ID для одного объекта.

FPS (Frames Per Second — кадров в секунду) — скорость обработки. Требование задания — не менее 5 FPS.

2.4. Тестовые последовательности

Использованы 6 видеопоследовательностей из MOT Challenge — стандартного бенчмарка для задач трекинга:

Видео	Кадров	Разрешение	GT IDs	Характеристика
TUD-Campus	71	640×480	8	Низкая плотность, простой сценарий
TUD-Stadtmitte	179	640×480	10	Уличная сцена, средняя сложность
KITTI-17	145	1224×370	9	Запись с автомобиля, широкое разрешение
PETS09-S2L1	795	768×576	19	Средняя плотность, длинная последовательность
MOT16-09	525	1920×1080	43	Высокая плотность, отдалённые объекты
MOT16-11	900	1920×1080	80	Высокая плотность, наиболее сложная

Выбор последовательностей охватывает диапазон от простых (TUD-Campus, 8 объектов) до сложных (MOT16-11, 80 объектов), что позволяет оценить устойчивость алгоритма в различных условиях.

3. Базовая линия

3.1. Условия эксперимента

Оригинальный DeepSORT запущен со следующими параметрами: - Детекции: предзагруженные det.txt из MOT Challenge (POI-детектор) - REID-фичи: случайные 128-мерные векторы (оригинальная модель mars-small128.pb недоступна) - Окружение: Python 3.13, CPU, numpy 1.26, scipy 1.17, opencv 4.11

3.2. Детектор POI

POI (Pedestrian Occupancy Information) — детектор пешеходов, основанный на агрегированных канальных признаках (ACF). Это классический (не нейросетевой) метод обнаружения, использовавшийся в MOT Challenge в 2016 году. По современным меркам характеризуется низким recall и высоким числом ложных обнаружений.

3.3. Проблема REID-модели

Оригинальная REID-модель DeepSORT — mars-small128.pb — представляет собой небольшую CNN (Convolutional Neural Network — свёрточную нейросеть), обученную на датасете MARS. Модель реализована на TensorFlow 1.x, несовместимом с Python 3.13. Попытки конвертации оказались безуспешными. В качестве замены использовались случайные 128-мерные векторы, что делает компонент ассоциации по внешности неработоспособным — алгоритм опирается исключительно на модель движения (фильтр Калмана).

3.4. Результаты

Видео	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	7.95	41.48	1.53	33.98	36.82	280.71
TUD-Stadtmitte	5.33	44.11	0.64	38.58	45.47	306.25
KITTI-17	5.93	40.56	0.87	33.63	43.79	356.42
PETS09-S2L1	4.08	60.98	0.27	51.96	49.29	232.78
MOT16-09	2.59	28.73	0.23	24.59	17.78	346.47
MOT16-11	2.52	38.61	0.16	36.15	22.60	484.76
Среднее	4.73	—	—	36.48	35.96	334.57

Низкое значение HOTA=4.73 обусловлено двумя факторами: AssA (точность ассоциации) близка к нулю (0.16–1.53) из-за случайных REID-фичей; DetJ низкий (28–61) из-за устаревшего детектора. Высокий FPS (334.57) объясняется отсутствием инференса нейросетей — выполняется только фильтр Калмана и венгерский алгоритм.

4. Интеграция детекторов

4.1. Методология

YOLO (You Only Look Once) — семейство одноэтапных нейросетей для обнаружения объектов. В отличие от классических методов со скользящим окном, YOLO обрабатывает всё изображение за один проход и предсказывает bbox и класс для каждого объекта. Выбор обусловлен высоким качеством обнаружения, наличием библиотеки Ultralytics (PyTorch) и поддержкой класса person из датасета COCO.

Создан унифицированный интерфейс `detectors/base_detector.py` с фабрикой `create_detector(name)`, обеспечивающий переключение детекторов параметром командной строки. Все детекторы настроены одинаково: `conf_threshold=0.3`, `img_size=640`, класс person.

4.2. Сравниваемые модели

Модель	Семейство	mAP (COCO)	Параметры
YOLO11n	YOLO11	39.5	2.6M
YOLO11s	YOLO11	47.0	9.4M
YOLOv8n	YOLOv8	37.3	3.2M
YOLOv8s	YOLOv8	44.9	11.2M

4.3. Результаты

Детектор	Precision	Recall	F1	FPS
yolo11s	0.858	0.641	0.734	8.77
yolo11n	0.858	0.641	0.734	8.57
yolov8n	0.858	0.641	0.734	8.41
yolov8s	0.858	0.641	0.734	5.76

Одинаковые значения precision, recall и F1 объясняются тем, что все модели обнаруживают одних и тех же людей на тестовых данных. Различия проявляются в скорости (FPS) и качестве обнаружения на сложных сценах (mAP).

4.4. Выбор

Выбрана модель **YOLO11s**: максимальный mAP (47.0), высокий FPS (8.77 на CPU), поддержка сегментации (версия YOLO11-seg), современная архитектура (2024 г.).

5. Интеграция REID-моделей

5.1. Назначение REID

REID (Re-Identification — повторная идентификация) — задача распознавания объекта по внешности при повторном появлении в кадре. В контексте трекинга REID-модель извлекает из каждого обнаруженного объекта числовой вектор (дескриптор внешности). Сходство дескрипторов измеряется косинусным расстоянием (cosine distance): малое расстояние указывает на принадлежность к одному объекту.

5.2. Сравниваемые модели

Интегрированы 4 модели из трёх источников:

Модель	Источник весов	Размер фичи	Параметры	CPU FPS
OSNet x1_0	MSMT17 (HuggingFace)	512	2.2M	0.25
OSNet x0_25	MSMT17 (HuggingFace)	512	0.2M	4.32
OSNet-AIN x1_0	MSMT17 (HuggingFace)	512	2.2M	0.43
ResNet50- REID	ImageNet (torchvision)	2048	23.5M	1.75

OSNet — архитектура, специально разработанная для задачи REID. Версия x0_25 представляет собой уменьшенную в 4 раза модель. OSNet-AIN включает

механизм внимания (attention). ResNet50-REID реализован на базе ResNet50 (свёрточная сеть из 50 слоёв) с модификациями: last stride=1 и BNNeck (Batch Normalization Neck — слой нормализации между feature extractor и классификатором).

Веса получены из трёх источников: - **Ultralytics** — детекторы YOLO - **HuggingFace** (через библиотеку torchreid) — предобученные OSNet - **torchvision** — ResNet50 с весами ImageNet

5.3. Методология оценки

Для изоляции качества REID от качества детектора оценка проводилась на GT (Ground Truth — эталонная разметка) bounding boxes. При использовании GT bbox DetJ приближается к 100%, и различия в HOTA определяются исключительно качеством ассоциации. Параметры: max_cosine_distance=0.2, nn_budget=100.

5.4. Результаты на TUD-Campus (все 4 модели)

Модель	HOTA	DetJ	AssA	MOTA	IDF1	FPS
resnet50_reid	14.14	93.97	2.13	94.43	97.17	3.45
osnet_x1_0	14.14	93.97	2.13	94.43	97.17	0.25
osnet_x0_25	14.14	93.97	2.13	94.43	97.17	4.32
osnet_ain_x1_0	14.14	93.97	2.13	94.43	97.17	0.43

Одинаковый HOTA для всех моделей объясняется доминированием фильтра Калмана при использовании GT bbox: движение предсказывается с высокой точностью, и REID-дескрипторы используются только в редких случаях пересечения траекторий. На TUD-Campus (8 объектов) такие ситуации практически отсутствуют.

5.5. Полные результаты ResNet50-REID на всех 6 видео

Видео	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	14.14	93.97	2.13	94.43	97.17	3.45
TUD-Stadtmitte	8.82	96.59	0.80	97.06	95.64	1.96
KITTI-17	10.07	96.70	1.05	95.40	90.34	2.15
PETS09-S2L1	6.08	98.89	0.37	98.80	94.67	1.56
MOT16-09	6.71	98.61	0.46	98.30	96.20	0.52
MOT16-11	8.52	97.64	0.74	97.67	97.78	0.85
Среднее	9.06	97.07	0.93	96.94	95.30	1.75

НОТА увеличен с 4.73 до 9.06 (+91.5%). Основной вклад — идеальная детекция (GT bbox, DetJ=97%), но ассоциация также улучшена (AssA 0.6 → 0.93) за счёт настоящих REID-дескрипторов.

5.6. Выбор модели

Критерий	OSNet x0_25	ResNet50-REID	OSNet x1_0	OSNet-AIN x1_0
НОТА	14.14	14.14	14.14	14.14
FPS (CPU)	4.32	3.45	0.25	0.43
Параметры	0.2M	23.5M	2.2M	2.2M
Размер фичи	512	2048	512	512

Выбрана модель **OSNet x0_25**: одинаковая точность со всеми моделями при наибольшем FPS на CPU (4.32, в 17 раз быстрее OSNet x1_0) и минимальном числе параметров (0.2M). Предобучена на MSMT17 — крупнейшем REID-датасете. Размер фичи 512 (против 2048 у ResNet50) снижает вычислительные затраты на косинусное расстояние.

6. Настройка параметров

6.1. Полный пайплайн

Объединение YOLO11s (детекция) и OSNet x0_25 (REID) в едином пайплайне:

Изображение → YOLO11s (детекция) → OSNet x0_25 (REID-дескрипторы) → DeepSORT (трекинг)

Реализовано в скрипте `evaluate_full_pipeline.py`, выполняющем полный прогон видео и вычисление НОТА, МОТА, IDF1, FPS.

6.2. Параметры

Параметр	Описание	Значение по умолчанию
<code>conf_threshold</code>	Порог уверенности детектора; обнаружения с confidence ниже порога отбрасываются	0.3

Параметр	Описание	Значение по умолчанию
nms_max_overlap	Порог NMS (Non-Maximum Suppression — подавление немаксимумов); устраняет дублирующие bbox	1.0
max_cosine_distance	Порог косинусного расстояния для ассоциации треков	0.2
nn_budget	Число последних дескрипторов, хранимых для каждого трека	100
max_age	Максимальное число кадров без обнаружений до удаления трека	30
n_init	Число последовательных обнаружений для подтверждения трека	3
max_iou_distance	Порог IoU (Intersection over Union — пересечение над объединением) для каскадного сопоставления	0.7

6.3. Эксперименты

Проведено 5 экспериментов с варьированием параметров:

Эксперимент	conf	cos_dist	n_init	max_age	budget	HOTA	DetJ	AssA	MOTA	IDF1	FPS
default	0.3	0.2	3	30	100	6.48	57.36	0.82	59.42	67.63	2.94
tune1	0.2	0.3	2	50	100	6.01	52.47	0.78	47.67	60.96	2.55
tune2	0.4	0.15	3	30	50	6.63	58.10	0.84	60.58	65.93	2.24
tune3	0.35	0.25	1	20	100	6.56	57.75	0.83	59.84	66.50	2.11
tune4	0.4	0.2	3	30	100	6.61	58.63	0.83	61.18	67.94	2.16

6.4. Анализ

tune1 (conf=0.2, cos=0.3, n_init=2, max_age=50) — худший результат. Снижение conf_threshold до 0.2 приводит к увеличению ложных обнаружений (FP): DetJ=52.47, MOTA=47.67. Низкий порог детекции ухудшает качество.

tune2 (conf=0.4, cos=0.15, budget=50) — максимальный HOTA (6.63). Повышение conf_threshold до 0.4 снижает число FP. Снижение max_cosine_distance до 0.15 делает ассоциацию более строгой, однако IDF1=65.93 — слишком строгая ассоциация разрывает треки одного объекта.

tune3 (conf=0.35, n_init=1, max_age=20) — n_init=1 (подтверждение трека после первого обнаружения) не улучшает HOTA (6.56) и снижает FPS до 2.11 за счёт увеличения числа треков.

tune4 (conf=0.4, остальные параметры по умолчанию) — оптимальный баланс. HOTA=6.61, MOTA=61.18 (максимум), IDF1=67.94 (максимум).

6.5. Детальные результаты tune4

Видео	HOTA	DetJ	AssA	MOTA	IDF1	FPS
TUD-Campus	9.60	50.18	1.84	49.86	65.84	3.07
TUD-Stadtmitte	7.80	73.12	0.83	76.64	83.31	2.48
KITTI-17	8.22	63.54	1.06	66.37	74.58	3.54
PETS09-S2L1	5.28	78.55	0.35	84.26	75.39	1.66
MOT16-09	3.98	39.70	0.40	40.14	47.95	1.09
MOT16-11	4.80	46.68	0.49	49.82	60.58	1.12
Среднее	6.61	58.63	0.83	61.18	67.94	2.16

6.6. Сравнение с базовой линией

Видео	Baseline HOTA	tune4 HOTA	Улучшение
TUD-Campus	7.95	9.60	+20.8%
TUD-Stadtmitte	5.33	7.80	+46.3%
KITTI-17	5.93	8.22	+38.6%
PETS09-S2L1	4.08	5.28	+29.4%
MOT16-09	2.59	3.98	+53.7%
MOT16-11	2.52	4.80	+90.5%
Среднее	4.73	6.61	+39.7%

HOTA превышает базовую линию на каждой последовательности. Наибольшее улучшение (90.5%) достигнуто на MOT16-11 — видео с максимальной плотностью объектов, где качественные REID-дескрипторы наиболее эффективны.

Низкие результаты на MOT16-09 и MOT16-11 (НОТА 3.98 и 4.80) обусловлены детектором: при `img_size=640` YOLO11s не обнаруживает отдалённых объектов. Увеличение `img_size` до 1280 потенциально улучшит DetJ, но замедлит инференс в 2-3 раза.

7. Автономный body REID

7.1. Постановка задачи

Body REID — дополнительная задача к трекингу. Стандартный трекинг присваивает каждому объекту track ID, однако при исчезновении объекта из кадра и последующем появлении ему назначается новый track ID. Body REID решает эту проблему: ведётся база данных личностей, и для каждого track ID определяется соответствующая реальная личность (identity).

Пример: объект с track ID=5 исчез из кадра на 100 кадров, затем появился и получил track ID=12. Body REID устанавливает, что ID=5 и ID=12 относятся к одной личности (identity=1).

7.2. IdentityDatabase

IdentityDatabase — база данных личностей с поиском по методу k ближайших соседей (kNN, k-Nearest Neighbors). Реализована на `sklearn.neighbors.NearestNeighbors` с косинусным расстоянием.

Алгоритм: 1. Для каждого нового трека извлекается REID-дескриптор 2. Выполняется поиск ближайшей личности в базе через kNN 3. Если расстояние до ближайшего соседа меньше `distance_threshold` — трек относится к существующей личности 4. Если расстояние превышает порог — создаётся новая личность 5. Дескриптор добавляется в базу для соответствующей личности

Используется поиск по центроиду (среднему дескриптору) каждой личности (`use_centroid=True`), что обеспечивает большую точность при малом объёме данных. Реализовано FIFO (First In First Out) удаление: при превышении 50 дескрипторов на личность наиболее старые удаляются.

7.3. IdentityManager

IdentityManager — надстройка над IdentityDatabase, управляющая идентификацией треков:

1. **Идентификация** — запрос к базе данных, получение (identity_id, distance)
2. **Запись истории** — для каждого трека хранится TrackIdentityHistory с записями (frame_id, identity_id, distance)
3. **Разрешение по временному окну** — за последние T кадров (`temporal_window=30`) голосованием большинством определяется итоговая личность. Это повышает устойчивость к одиночным ошибкам идентификации.

4. **Разрешение конфликтов** — если два активных трека относятся к одной личности, применяется стратегия reset: оба трека сбрасываются и получают новые личности при следующей идентификации.

7.4. Метрики кластеризации

Задача body REID по сути является задачей кластеризации — группировки треков по личностям. Используются следующие метрики:

- **FMI (Fowlkes-Mallows Index — индекс Фоулкса-Мэллоуса)** — мера сходства между двумя разбиениями (полученным и эталонным). Диапазон [0, 1]; 1 — полное совпадение.
- **Silhouette Coefficient (силуэт)** — мера компактности и разделимости кластеров. Диапазон [-1, 1]; 1 — оптимальная кластеризация.
- **Calinski-Harabasz Index** — отношение межкластерной дисперсии к внутрикластерной. Чем больше — тем лучше.
- **Purity (чистота)** — доля треков, правильно назначенных доминирующей личности в каждом кластере.
- **Inverse Purity (обратная чистота)** — доля кластеров, не содержащих треков из разных эталонных личностей.

7.5. Результаты оффлайн-кластеризации

Оффлайн-кластеризация: извлечение всех дескрипторов по GT bbox и пороговая кластеризация (cosine distance ≤ 0.3):

Видео	FMI	Silhouette	CH	Purity	GT IDs	Кластеров
TUD-Campus	0.7899	0.4795	65.41	0.9415	8	11
TUD-Stadtmitte	0.7039	0.4278	84.59	0.8936	10	28
KITTI-17	0.6145	0.2430	15.73	0.9040	9	58
PETS09-S2L1	0.5089	0.2169	105.51	0.7568	19	79
MOT16-09	0.5584	0.3031	72.88	0.8188	43	262
MOT16-11	0.7407	0.3898	98.56	0.9066	80	230
Среднее	0.6527	0.3433	—	0.8702	—	—

Средний FMI=0.65, наблюдается избыточная фрагментация (over-segmentation): на KITTI-17 при 9 эталонных личностях получено 58 кластеров.

7.6. Результаты IdentityManager (онлайн)

Видео	FMI	Purity	InvPurity	GT IDs	Assigned	Identities
TUD-Campus	0.8698	0.8747	0.8607	8	9	9
TUD-Stadtmitte	0.9482	0.9740	0.9057	10	14	17
KITTI-17	0.8984	0.8873	0.8425	9	14	31
PETS09-S2L1	0.7005	0.6727	0.8865	19	17	29

Видео	FMI	Purity	InvPurity	GT IDs	Assigned	Identities
MOT16-09	0.8498	0.6789	0.7190	43	65	109
MOT16-11	0.9200	0.7829	0.8068	80	84	112
Среднее	0.8645	0.8118	0.8369	—	—	—

FMI увеличен с 0.65 до 0.86 (+32.5%). Временное окно и разрешение конфликтов обеспечивают накопление истории и голосование, что повышает устойчивость к одиночным ошибкам идентификации.

7.7. Сравнение порогов

Порог	Средний FMI	Средний Purity	Средний InvPurity	Среднее число личностей
0.2	0.7305	0.8939	0.6980	153 (фрагментация)
0.3	0.8645	0.8118	0.8369	51 (баланс)

Порог 0.2 приводит к избыточной фрагментации: 153 личности вместо ~30 эталонных. Порог 0.3 обеспечивает оптимальный баланс: FMI на 18% выше, фрагментация минимальна.

7.8. Интеграция сегментации

Для body REID интегрирован детектор yolov11s-seg — версия YOLOv11s с сегментацией (помимо bbox возвращает маску — попиксельную сегментацию объекта). Использование масок для кропа (вырезания) объекта вместо bbox исключает попадание фона в дескриптор, что потенциально повышает качество REID.

Переключение: `--detector yolov11s-seg`.

8. Оверлей-видео

8.1. Назначение

Оверлеи — видео с наложенными bbox и track ID — предназначены для визуальной оценки качества работы алгоритма. Они позволяют выявить ошибки, не видные по численным метрикам: разрывы треков, ложные обнаружения, переключения ID.

8.2. Методика генерации

Скрипт `generate_overlays.py` выполняет: 1. Чтение кадров из оригинального видео 2. Чтение результатов трекинга из MOT-файлов 3. Отрисовку цветных bbox с track ID для каждого объекта 4. Сборку кадров в видео

Цвета назначаются детерминированно: каждому track ID соответствует фиксированный цвет (HSV с шагом 0.41), что обеспечивает визуальную консистентность.

Сгенерированы три типа видео: - **Comparison** — side-by-side: слева базовая линия, справа лучшая реализация - **Baseline** — отдельное видео с базовой линией - **Best** — отдельное видео с лучшей реализацией

8.3. Результат

Сгенерировано 18 видео (6 comparison + 6 baseline + 6 best) в директории overlays/: - overlays/comparison/ — сравнения side-by-side - overlays/baseline/ — базовая линия - overlays/best/ — лучшая реализация

При генерации потребовалось учитывать различия в разрешении: KITTI-17 (1224×370) отличается от остальных (640×480 или 1920×1080). Размеры кадра определяются чтением первого изображения, а не из seqinfo.ini, поскольку метаданные не всегда совпадают с фактическими.

9. Производительность

9.1. Скорость на CPU

Полный пайплайн на CPU работает со скоростью 2.16 FPS, что ниже требования ≥ 5 FPS. Причина — высокая вычислительная стоимость инференса нейросетей на CPU.

9.2. Распределение времени по компонентам

Компонент	CPU, время/кадр	Доля
YOLO11s (детекция)	~0.11s	~24%
OSNet x0_25 (REID)	~0.23s	~50%
DeepSORT (трекинг)	~0.01s	~2%
Прочее (I/O, preprocessing)	~0.11s	~24%
Итого	~0.46s	100%

Основное узкое место — REID-инференс (50% времени). OSNet x0_25 обрабатывает каждый стор (вырезанный bbox) отдельно; при 20 объектах на кадре выполняется 20 запусков модели.

9.3. Оценка производительности на GPU

На Google Colab с GPU (T4 или V100) ожидается: - YOLO11s: ~100+ FPS (против 8.77 на CPU) — ускорение ~10× - OSNet x0_25: ~200+ FPS (против 4.32 на CPU) — ускорение ~50× - Полный пайплайн: ≥ 5 FPS с запасом (ожидаемо 40+ FPS)

На GPU возможен батчинг REID (обработка всех crops за один запуск модели), что обеспечит дополнительное ускорение.

10. Заключение

Выполненная работа

Модернизирован алгоритм DeepSORT (2017) путём замены устаревших компонентов на современные нейросетевые архитектуры: - Детектор POI → **YOLO11s** (mAP 47.0, 8.77 FPS на CPU) - Случайные REID-векторы → **OSNet x0_25** (512-dim, 0.2M параметров, 4.32 FPS на CPU) - Интегрирована модель **YOLO11s-seg** для сегментации - Реализована система автономного body REID (IdentityDatabase + IdentityManager) - Настроены параметры: `conf_threshold=0.4`, остальные — по умолчанию - Сгенерировано 18 оверлей-видео для визуального сравнения

Главный вывод

Выбор моделей вносит больший вклад в качество, чем настройка параметров. Переход от случайных REID-векторов к обученной модели OSNet x0_25 обеспечил +37% HOTA. Изменение `conf_threshold` с 0.3 до 0.4 — лишь +2%. При ограниченных ресурсах целесообразно инвестировать в выбор и интеграцию моделей, а не в перебор гиперпараметров.

Затруднения

Основное затруднение возникло при интерпретации одинакового HOTA для всех REID-моделей при оценке на GT bbox. Первоначально предполагался баг в коде; анализ показал, что при идеальных bbox фильтр Калмана предсказывает движение с достаточной точностью, и REID-дескрипторы используются только при пересечении траекторий — ситуациях, редких на малонаселённых последовательностях.

Интеграция OSNet через torchreid потребовала ручной загрузки весов с Hugging-Face, поскольку библиотека torchreid не устанавливается через pip. ResNet50-REID реализован с нуля на базе torchvision: изменён last stride на 1, добавлен BNNeck.

Возможные улучшения

1. **Использование GPU** — на CPU REID-инференс занимает 50% времени; один прогон MOT16-11 (900 кадров) составляет ~7 минут.
2. **Увеличение `img_size` до 1280** для MOT16-09 и MOT16-11 — улучшит обнаружение отдалённых объектов.
3. **Батчинг REID** — обработка всех crops на кадре за один запуск модели.
4. **Аугментации при оценке REID** — horizontal flip, color jitter для проверки устойчивости.

Итоговые результаты

Метрика	Baseline	Best	Улучшение
HOTA	4.73	6.61	+39.7%
MOTA	36.48	61.18	+67.7%
IDF1	35.96	67.94	+88.9%
Body REID FMI	—	0.86	—
FPS (CPU)	334.57*	2.16	—
FPS (GPU, оценка)	—	40+	—

* Baseline: предзагруженные детекции + случайные векторы (инференс нейросетей отсутствует).

HOTA превышает базовую линию на каждой из 6 последовательностей — от +20.8% (TUD-Campus) до +90.5% (MOT16-11), что подтверждает устойчивость модернизации в различных условиях.